Transcript for Phase 1 Presentation
Lightweight Spike-Based DOA Sector Inference for Embedded Edge AI

Hossein Molhem
Kennesaw State University
EE 6900 Embedded AI

July 2, 2026

Slide 1 — Title

In this presentation, I summarize Phase 1 of my embedded-AI project. The project is titled lightweight spike-based DOA sector inference for embedded edge AI. The purpose is to connect a physics-based sensing model with spike-based temporal inference and to prepare the project for later embedded or hardware-aware implementation.

Slide 2 — Motivation

The motivation comes from edge sensing systems. These systems often cannot rely on large models or expensive computation because they may have limited power, memory, and latency budgets. Spike-based inference is interesting in this context because it represents information through sparse temporal events. For the sensing problem, I selected DOA sector classification because it is simple enough for a focused prototype but still grounded in real array signal processing.

Slide 3 — Phase 1 Objective

The objective of Phase 1 is to build a reproducible Python reference prototype. The prototype uses a uniform linear array signal model, extracts beamspace features, applies rate-based spike encoding, and performs classification using a LIF-inspired temporal classifier. The goal is not to claim a new DOA algorithm. The goal is to establish a controlled engineering baseline that can later be refactored toward PYNQ, FPGA, or Vitis HLS implementation.

Slide 4 — End-to-End Pipeline

The pipeline starts with a ULA-based DOA signal model. From that model, array snapshots are generated at controlled SNR values. A sample covariance matrix is computed from the snapshots. Beamspace features are then extracted from the covariance matrix and normalized. These normalized features are used in two ways: first, by a nearest-centroid baseline classifier, and second, by a spike-encoded LIF-inspired classifier.

Slide 5 — Signal Model

The signal model is a narrowband single-source ULA model. The received array snapshot matrix is written as $X = a(\theta)s + N$, where $a(\theta)$ is the steering vector, s is the source signal, and N is complex Gaussian noise. The covariance matrix is computed as $R_{xx} = XX^H/M$, with $M = 128$ snapshots. This covariance matrix is the basis for extracting compact beamspace features.

Slide 6 — Dataset Configuration

The dataset uses eight array elements, half-wavelength spacing, and 128 snapshots per sample. The source angle ranges from negative 60 degrees to positive 60 degrees and is divided into four DOA sectors. The SNR values range from negative 10 dB to 20 dB. The total dataset contains 2800 samples, with 840 samples used for testing after a stratified train-test split.

Slide 7 — Beamspace Response Example

This slide shows an example beamspace response. The important point is that the beamspace representation provides compact spatial information. Instead of estimating the exact continuous angle, Phase 1 classifies the signal into one of four sectors. This makes the problem suitable for a lightweight embedded inference prototype.

Slide 8 — Baseline and LIF Inference

There are two classification paths. The first is the nearest-centroid baseline, which operates directly on continuous normalized beamspace features. It is deterministic and interpretable, and it achieved an overall accuracy of 0.869048. The second path converts the features into rate-coded spike trains and classifies them using a LIF-inspired temporal evidence accumulator. The fixed Run 003 LIF configuration achieved 0.825000 accuracy.

Slide 9 — LIF Temporal Accumulation

The LIF-inspired classifier accumulates evidence over time using the update equation $v_c[t] = \alpha v_c[t-1] + i_c[t]$. Here, $v_c[t]$ is the membrane evidence for class c, α is the leak factor, and $i_c[t]$ is the spike-driven input current. The stable-decision latency is defined as the earliest time step after which the final class decision remains unchanged. In the fixed Run 003 setting, the time window was 20 steps, the leak was 0.97, the accuracy was 0.825000, and the mean latency was 6.423810 time steps.

Slide 10 — Accuracy Comparison

The accuracy comparison shows the continuous nearest-centroid baseline and the LIF-inspired classifier across the same SNR values. As expected, the continuous baseline performs better. However, the LIF classifier remains far above the 25 percent random baseline for four balanced classes. This shows that the sparse spike representation still preserves meaningful sector-level DOA information.

Slide 11 — Latency Versus SNR

This slide shows the mean stable-decision latency of the LIF classifier versus SNR. The key observation is that higher signal quality generally allows the temporal decision to stabilize earlier. This supports the edge-inference interpretation because a future embedded implementation may exploit this behavior for early-exit or adaptive-latency decisions.

Slide 12 — Run 005 Parameter Sweep

Run 005 swept the spike time-window length and the membrane leak factor. The best LIF accuracy was 0.841667 at 40 time steps and leak 0.99. The lowest latency configuration used 10 time steps and leak 0.99, with accuracy 0.776190 and mean latency 3.782143 time steps. This comparison shows the direct trade-off between accuracy and latency.

Slide 13 — Accuracy–Latency Trade-Off

The trade-off plot summarizes the behavior of the LIF parameter sweep. Longer spike windows and stronger evidence retention generally improve accuracy because the rate-coded representation has more time to approximate the original continuous feature vector. At the same time, longer windows increase the temporal budget. This is exactly the type of design knob that matters for embedded AI.

Slide 14 — Engineering Interpretation

The main engineering interpretation is that the beamspace features are informative, the spike encoding introduces a moderate but understandable accuracy loss, and the LIF parameters provide explicit control over the accuracy-latency trade-off. This makes Phase 1 a useful reference model rather than just a toy example. It is now stable enough to support later hardware-aware refactoring.

Slide 15 — Conclusion and Next Steps

In conclusion, Phase 1 successfully establishes an end-to-end DOA/SNN reference prototype with reproducible metrics, figures, and documentation. The immediate next step is to finalize the course report package. After that, the project can be extended toward richer radar and DOA case studies, and selected kernels can be refactored for PYNQ or Vitis HLS.